

# 56. Devant, derrière : le tuyau en une ligne, `spriteDerriere()`

Le tuyau du 110.4, avec `spriteDerriere(0, x, 72, HEROS)` : plus de 5e argument à retenir.

## 1. Ce qu'il faut comprendre

**Le programme du tuyau (le 110.4), avec un seul changement :** `sprite(0, x, 72, HEROS, DERRIERE)` devient `spriteDerriere(0, x, 72, HEROS)`.

**Ce qui est nouveau ici :** `spriteDerriere(numero, x, y, tuile)`, une fonction de la console. Le nom dit ce qu'elle fait : on n'a plus à se souvenir que `DERRIERE` se met en 5e position.

**Le même prix qu'un `sprite()`** : le compilateur la remplace par `sprite(..., DERRIERE)`. On peut l'appeler à chaque image, dans la boucle du jeu, sans rien alourdir.

**Et pour le repasser devant ?** On rappelle `sprite()` tout court, sans 5e argument : chaque appel range le lutin de nouveau, avec ce qu'on lui donne.

**À toi :** fais-le passer devant le tuyau à l'aller, derrière au retour. Une variable `sens` et un `if` suffisent : `if (sens == 0) sprite(...); else spriteDerriere(...);`.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **506 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                    |       |                                                                       |
|--------------------|-------|-----------------------------------------------------------------------|
| démarrage          | 57 o  | ← la console, à l'allumage — la mise en route de la console           |
| main()             | 103 o | ← démarrage — ta fonction                                             |
| VBlank             | 8 o   | ← la console, à chaque image — compte les images, 60 fois par seconde |
| AttendreImage      | 28 o  | ← image() — attend l'image suivante                                   |
| AttendreVBlank     | 16 o  | ← démarrage — attend que l'écran soit libre                           |
| AttendreAcces      | 12 o  | ← EcrireTexte — attend de pouvoir écrire une case                     |
| AdresseCase        | 10 o  | ← poser() — trouve l'adresse d'une case de l'écran                    |
| EcrireTexte        | 10 o  | ← texte() — écrit un texte, lettre par lettre                         |
| RangerLutins       | 11 o  | ← démarrage — range les 40 lutins hors de l'écran                     |
| InstallerTransfert | 15 o  | ← démarrage — installe la copie des lutins en mémoire rapide          |
| CopierTuiles       | 76 o  | ← démarrage — copie les lettres et les dessins en mémoire vidéo       |
| EffacerCarte       | 14 o  | ← démarrage — vide les deux cartes de l'écran                         |
| Tuiles             | 128 o | ← CopierTuiles — 6 lettres sur 43, et 2 tuiles dessinées              |
| DonneesTransfert   | 10 o  | ← InstallerTransfert — la petite routine qui copie les lutins         |
| "LE TUYAU"         | 8 o   | ← texte() — un texte gravé                                            |

Lettres gravées : AELTUY

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

|                       |     |                                                  |
|-----------------------|-----|--------------------------------------------------|
| <code>sprite()</code> | × 1 | 24 o sur place, dans main()                      |
| <code>poser()</code>  | × 1 | 13 o sur place, dans main() + AdresseCase (10 o) |
| <code>texte()</code>  | × 1 | 11 o sur place, dans main() + EcrireTexte (10 o) |

```
image()  x 1      6 o sur place, dans main() + AttendreImage (28 o)
images() x 1      2 o sur place, dans main()
```

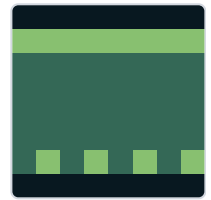
## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : le tuyau du 110.4. Le héros y entre et ressort de l'autre côté.
//
// Ce qui est nouveau : spriteDerriere(numero, x, y, tuile).
//
// AVANT                                MAINTENANT
// sprite(0, x, 72, HEROS, DERRIERE);    spriteDerriere(0, x, 72, HEROS);
//
// Le compilateur remplace la ligne de droite par celle de gauche : même prix.

#include <Tuile>           // un dessin de 8 × 8 pixels
#include <poser>           // pose une tuile sur une case du fond
#include <texte>           // écrit un texte à l'écran
#include <spriteDerriere> // place un lutin derrière le décor
```

## 2. UN DESSIN — TUYAU

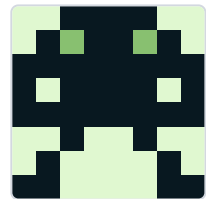
```
Tuile TUYAU = {                // Un morceau de tuyau couché : AUCUN 0, il est tout plein.
    "33333333",
    "11111111",
    "22222222",
    "22222222",
    "22222222",
    "22222222",
    "21212121",
    "33333333",
};
```



à l'écran

## 3. UN DESSIN — HEROS

```
Tuile HEROS = {                // Le héros : les points (.) sont transparents.
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};
```



à l'écran

## 4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                    // Le jeu commence ici.
    texte(1, 1, "LE TUYAU");

    for (uint8_t c = 8; c < 13; c++) { // Le tuyau : colonnes 8 à 12, ligne 9.
        poser(c, 9, TUYAU);
    }

    uint8_t x = 0;              // La place du héros, en pixels.

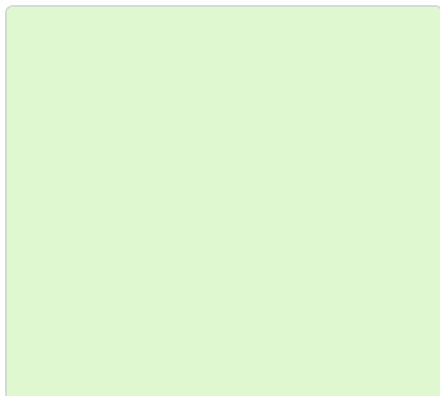
    while (true) {              // La boucle du jeu :
        image();

        if (images() % 2 == 0) x++; // Un pixel toutes les 2 images.
        if (x > 152) x = 0;

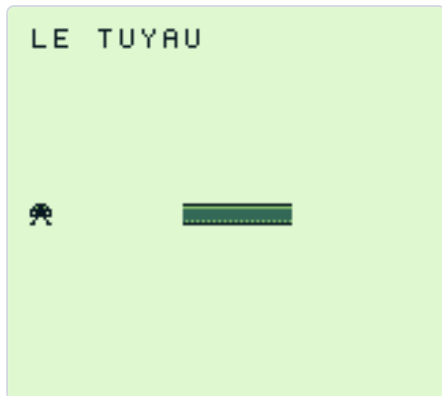
        spriteDerriere(0, x, 72, HEROS); // Le héros, derrière le décor : une ligne, sans 5e argument.
    }
}
```

### 3. Ce qui se passe, étape par étape

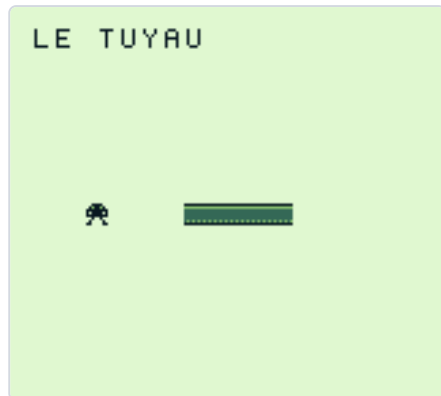
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



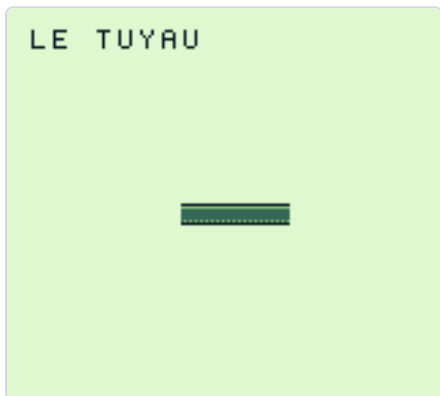
la toute première image — la console vient de démarrer — image 1



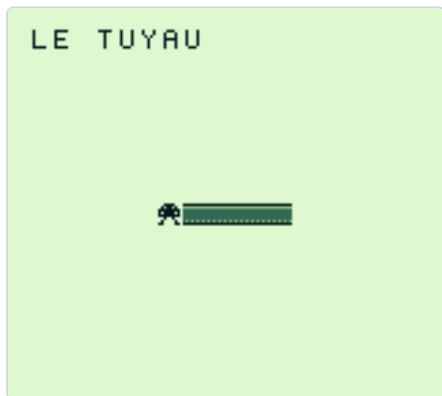
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

### 4. Ce qu'on doit voir

Le même écran qu'au 110.4 : le héros disparaît dans le tuyau et ressort à droite.

### 5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ sur le vide, on voit le héros (nuance 3)
- ✓ dans le tuyau, on voit le tuyau (nuance 2)
- ✓ le lutin porte bien DERRIERE

← [55. Devant, derrière : le buisson sur Game Boy Color, poserDevant\(\)](#)

[sommaire](#)

[57. Deux boîtes qui se touchent](#) →